TD 3 – Objets 3D existants – Importation, manipulation et animation

Introduction

Dans les TDs précédents, vous avez appris à créer une scène 3D de toute pièce à l'aide de ThreeJS en ajoutant des éléments 3D simples, en les groupant, les associant, ainsi qu'en définissant un certain nombre d'éléments spécifiques tel que : Camera, Renderer, Lights, tous nécessaires à la création de la scène.

Vous avez également vu comment créer une animation rudimentaire.

ThreeJS offre de nombreuses possibilités complémentaires, grâce à un certain nombre d'extensions.

Dans le cadre d'un intranet, il pourrait être nécessaire par exemple de charger des modèles 3D d'objets préconçus avec un logiciel tiers, en vue de la mise à disposition pour visualisation.

Avec ThreeJS, il est entre autres possible de charger et manipuler des objets préexistants, conçus et exporter avec des modeleurs 3D tel que Blender 3D, grâce à l'une de ses extensions.

Il est également possible de manipuler les animations contenues dans le fichier binaire exporté.

Formats de fichiers et usages

Les fichiers 3D sont utilisés dans de nombreux domaines parmi lesquels les jeux-vidéos, les films ou encore la conception assistée par ordinateur. Ils permettent de stocker les informations relatives à des modèles ou objets 3D en encodant les différentes informations de ces derniers, de la forme au mouvement en passant par la couleur, la texture ou encore la source de lumière.

Il existe de nombreux formats de fichier 3D, adaptés à différents usages, qu'ils incluent de nombreuses fonctionnalités ou seulement une, qu'il s'agisse de format propriétaire ou non.

Voici ci-dessous 3 de ces formats (il en existe de nombreux autres), ayant chacun leurs spécificités mais qui ont trouvé leur place parmi les autres de par leur capacité à fonctionner sur de nombreux logiciels et supports informatiques différents.

.obj (Wavefront 3D Object File)

L'OBJ (.obj) est un format ouvert et universellement reconnu, initialement développé par la société Wavefront Technologies. Structuré en ASCII, il contient la description des données géométriques d'un modèle 3D (sommets, coordonnées de texture, couleur). Il ne peut cependant pas supporter toutes les informations comme celles relatives à l'animation du modèle 3D. Le .obj est donc privilégié pour un travail en local ou pour des exportations d'images non animées.

Avec le développement de l'impression 3D, le format OBJ connaît un regain d'énergie notamment parce qu'il prend en compte l'affichage des couleurs, ce que le format STL, principalement utilisé dans ce domaine, ne permet pas.

Parmi les logiciels supportant le OBJ, on pourrait citer Maya, Blender, 3D Studio Max ou encore MeshLab.

.x3d (Extensible 3D)

Le format X3D est un format ouvert de fichier 3D basé sur le XML, créé par le consortium Web3D dans le but de supplanter le VRML (Virtuality Reality Modeling Language).

Le format X3D reprend l'ensemble des spécificités du format VRML (couleur, texture, transparence, etc.) mais a été repensé de manière a être adapté au web et notamment dans le but de pouvoir être lancé depuis un navigateur grâce notamment à OpenGL.

.glTF (GL Transmission Format)logo du format d'échange glTF

Le format glTF 2.0 est un format ouvert, introduit pour améliorer et accélérer les échanges de données 3D entre les différents outils et services de contenus 3D.

À l'instar du COLLADA (un autre type en perte de vitesse par rapport à glTF), il enregistre toutes les données relatives à l'objet 3D : géométrie, maillages, textures, matériaux, hiérarchie des nœuds, lumières, caméras, animations et autres propriétés.

Il couramment utilisé sur le Web et prend en charge divers moteurs 3D tels que Unity3D, Unreal Engine et Godot.

Aujourd'hui de nombreux logiciels de CAO ou de modélisation permettent d'exporter vos modèles 3D en glTF, dont SolidWorks, 3dsmax, Maya et Revit ou Blender.

Etape 1 : Mise en place d'une scène de base pour recevoir l'objet 3D à charger

Le chargement d'un objet 3D conçu avec un modeleur dépend fortement de son type.

De son côté ThreeJS met à votre disposition plusieurs « extensions » appelées ***Loader***, permettant de charger plusieurs des types disponibles.
Il convient donc, pour pouvoir charger un modèle 3D, de connaitre son type afin de sélectionner le bon *Loader.*

Dans les ressources fournies vous avez à disposition 2 objets 3D de type « GL Transmission Format » (GTlf).
Il est disponible sous 2 formes : une décompressée, lisible au format Json mais plus lourde, avec pour extension GTlf, et une seconde au format binaire, empaquetant tout le nécessaire y compris des animations si elles existent pour l'objet 3D dans un seul fichier, avec pour extension GLB.

C'est ce dernier format que vous allez utiliser, les 2 ressources fournies étant de ce type.

Préparez le code nécessaire à la mise en œuvre d'une scène 3D, comme vu dans le TD précédent, en y prévoyant une ou plusieurs sources lumineuses, une caméra, un renderer, mais sans objet ni transformation (rotation) associé, vous allez l'ajouter sous peu.

Etape 2 : Chargement de l'objet 3D externe

Une fois votre scène prête, plusieurs étapes sont nécessaires pour charger votre modèle (ici le perroquet modélisé dans le fichier nommé parrot.glb).

Pour cela vous allez devoir utiliser la classe *Loader* ThreeJs dédié, appelée tout naturellement GLTFLoader, contenu dans un addon de ThreeJs à importer dans le fichier.

Ajoutez les lignes ci-dessous dans votre script suivant pour charger le package contenant l'addon GLTFLoader, et profitez-en également pour charger l'addon OrbitsControls qui permet d'obtenir un contrôle de la position de la caméra à l'aide de la souris :

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
```

Cette classe met à disposition plusieurs méthodes, dont la méthode de chargement, nommée load.
Cette méthode prend 4 paramètres :

- Le chemin vers le fichier contenant l'objet / la scène 3D à charger
- Une fonction de callback dont le paramètre sera un pointeur vers l'objet chargé, donnant accès à ce dernier à l'intérieur de la fonction, exécuté à la fin du chargement
- Une fonction de callback appelée tant que le chargement progresse (le paramètre qu'elle peut prendre permet d'obtenir des données sur le chargement et notamment de calculer un pourcentage de complétion)
- Une fonction de callback appelée en cas d'erreur lors du chargement

```
const loader = new GLTFLoader();  // attention
loader.load( '/cube/parrot.glb', function ( parrot ) {  });
```

Ici le pointeur référençant l'objet chargé est nommé parrot (entre paranthèse de la fonction).
Cette fonction de callback permet de spécifier, une fois le modèle 3D chargé, un certain nombre de paramètres de positions, de rotations, etc… grâce aux propriétés de l'objet chargé passé en référence de la fonction (ici parrot).

Vous allez modifier légèrement la dimension du modèle à l'aide de *scale* ainsi que son éclairage ambiant et sa position, pour le centrer dans la scène (pour rappel, si vous avez utilisé les mêmes réglages de scène que pour le premier TD, la caméra « regarde » en 0,0,0).

Attention : il est souvent nécessaire de modifier la valeur d'échelle (*scale*) de l'objet car selon le modèle chargé, selon qui l'a créé, et selon l'outil utilisé, sa dimension par défaut peut être très variable, une fois le chargement terminé.

```
loader.load( '/cube/parrot.glb', function ( gltf ) {
                                    gltf.scene.position.set(0,0,0);
                                    gltf.scene.AmbientLight = new THREE.AmbientLight( 0xdddddd, 0.8 );
                                    gltf.scene.scale.set(1,1,1);
                                    scene.add( gltf.scene );
                                    // ---------- STARTING THE RENDER LOOP ----------
                                    toRender(); // this method render the scene at each frame
                            },
```

```
// called while loading is progressing
function ( xhr ) {
        console.log( ( xhr.loaded / xhr.total * 100 ) + '% loaded' );
},
// called when loading has errors
function ( error ) {
        console.log( 'An error happened' );
});
```

On peut voir dans le code précédent les 3 fonctions de callback attendues en paramètre :

- la première effectuant un paramétrage de position et de dimension en utilisant comme vu plus haut la référence à l'objet chargé, *parrot*,
- la seconde, appelée lors du chargement, avec le paramètre nommée *xhr* disposant de deux propriétés : *loaded* qui contient le nombre d'octets chargés et *total* qui donne la taille total en octets, rapporter dans la console du navigateur en pourcentage du total dans le code ci-dessus,
- la troisième, qui permet d'effectuer un traitement particulier (affichage, log ou autre) en cas d'erreur lors du chargement, dans le cas ci-dessus, un affichage dans la console du navigateur.

Lors du rafraichissement de la page, vous pouvez voir votre modèle 3D chargé, représentant un perroquet.

Exercice :

En utilisant un dépôt comme https://sketchfab.com/feed, tentez dans un nouveau fichier (il est important de garder l'actuel pour la suite du TD), de charger un modèle de votre choix, et d'effectuer un rendu de celui-ci dans votre navigateur.

Etape 3 : Animation d'un modèle existant (qui en contient)

Lorsque qu'un modeleur 3D crée un modèle, selon l'outil modeleur utilisé, il peut créer une ou plusieurs animations liées à ce modèle, et, lorsqu'il exporte son modèle, selon le format choisi, (c'est le cas du format GLB), les animations peuvent être enregistrées dans le fichier.

C'est le cas du fichier parrot.glb fourni, et vous allez déclencher ces animations dans votre script afin d'obtenir un perroquet chargé et animé.

Pour cela, ThreeJS fournit un ensemble de classe dans un module, permettant de créer et jouer des animations (créées, ou récupérées dans un fichier), **AnimationMixer**.

Vous allez charger au passage **Clock**, module qui permet de manipuler un timer, qui va vous être utile pour jouer l'animation.

Ajoutez la ligne suivante dans votre fichier, et déclarez 2 variables globales pour manipuler ces deux objets un peu plus tard (par exemple, *clock* et *mixer*).

```
import { Clock , AnimationMixer } from 'three';
```

Dans la partie création de la scène générale, ajoutez également le code suivant qui initialise ce timer, ainsi que le contrôle à la souris, évoqué un peu plus tôt :

```
// Load the Orbitcontroller
controls = new OrbitControls( camera, renderer.domElement );
controls.update();


clock = new Clock() ;
```

Maintenant que votre MixerAnimation est chargé, vous allez instancier un objet de cette classe, qui prend en paramètre de son constructeur une référence à l'objet que l'on souhaite animer ici *parrot* (s'il comporte une animation disponible).
Cette animation, est accessible via la référence de l'objet (uniquement disponible dans la première fonction callback passée en paramètre du loader), c'est donc là qu'il faut instancier votre MixerAnimation.

```
//Prepare animation
mixer = new AnimationMixer( parrot.scene );
```

Cet objet *mixer* dispose de plusieurs méthodes dédiées à l'animation, dont une en particulier, appelée clipAction :

- qui prend en paramètre une animation, récupérable dans l'objet chargé, via un tableau nommé ***animations*** qui contient toutes les animations disponibles si elles existent,
- qui renvoie un objet correspondant à l'animation voulue, cette dernière pouvant être manipulée grâce à plusieurs méthodes, dont ***play()***, qui permet de la lancer.

```
//and play it
mixer.clipAction( parrot.animations[ 0 ] ).play();
```

Enfin, pour que l'animation puisse être vue étape par étape et pas en une seule fois (c'est-à-dire qu'on ne verrait rien) lors du chargement de l'objet, vous allez utiliser le timer dans la fonction de rendu toRender() (la fonction qui gère le rendu, frame par frame, créé lors du premier TD) , en y ajoutant :

```
var delta = clock.getDelta();
mixer.update( delta );
controls.update();
```

La première ligne sert à récupérer un delta de temps écoulé depuis la dernière frame rendue, qu'il faut passer à la méthode update de votre objet mixer (qui gère l'animation).

Notez au passage, l'appel à la méthode *update()* de l'objet *controls* créé un peu plus tôt, qui va vous permettre de modifier l'angle de camera dans la scène à l'aide de la souris.

Exercice :

Ajouter un arbre (fourni : oak_tree.glb) dans cette scène et utilisez vos connaissances acquises pour obtenir un perroquet volant et tournant autour de l'arbre.